The mathematical model of a small working robot

Mini-IPHOREOS: belief estimation, short-pulse control, verification and engineering realisation



AI-generated concept illustration; not a photograph or an assembly drawing.

One observable task. Approach a cardboard wall and verify that the ultrasonic sensor face is within 21–29 cm of it, with a nominal target of 25 cm.

This companion derives the equations, documents the implemented decisions, explains calibration, and connects every stage to the practical build tutorial. It distinguishes implemented behaviour from possible research extensions.

Start with: the illustrated A–Z tutorial or the project homepage. Keep this PDF beside those files so relative links resolve. Some PDF readers require permission to open local links; the printed chapter numbers remain usable.

Prepared for Florent Ouabo Kamkumo
10 September 2026 / Implementation companion v1.0
Status: computer-tested software; physical calibration and validation pending.

Contents

1 The task, the mathematical question and the answer **2**

2 Variables, units and the complete internal state **2**

3 From ultrasound time to a distance observation **3**

4 The movement model and its physical meaning **4**

5 Deriving the scalar Bayesian update **4**

6 Observation guards and their exact scope **5**

7 Verification: what DONE means mathematically **5**

8 The implemented Mini movement decision **6**

9 Three policies and the role of context **7**

10 Worked calculations you can reproduce **8**

10.1 A prediction–measurement cycle 8

10.2 Same measurement, different context 8

10.3 Three confirmations 8

10.4 Confidence can be wrong 8

11 Calibration as parameter estimation **8**

11.1 Sensor bias and measurement variance 8

11.2 Travel per pulse 9

11.3 Identifiability and model checking 9

12 From the mathematical action to motor power **9**

13 Default parameters: a complete reproducible specification **10**

14 Simulation: what the reported results actually measure **11**

15 Physical experiment design and acceptance evidence **12**

16 Reading route, source files and tutorial crosswalk **13**

1 The task, the mathematical question and the answer

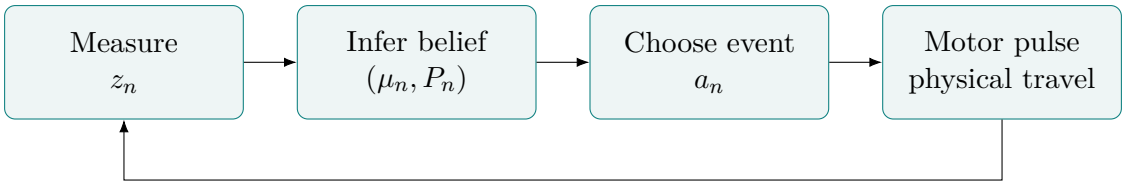
The mathematical question is: *given an uncertain distance measurement and an imperfect movement model, should the robot move a little, measure again, or stop?* The answer implemented here is a scalar Bayesian estimator inside a bounded state machine.

The physical objective uses true distance d, not the sensor output:

$$d_\star = 25 \text{ cm}, \qquad \varepsilon = 4 \text{ cm}, \qquad \mathcal{T} = [d_\star - \varepsilon, d_\star + \varepsilon] = [21, 29] \text{ cm}. \tag{1}$$

Success means the controller reports `DONE` and an independent measurement confirms $d \in \mathcal{T}$. A happy-face display alone establishes only the controller's conclusion.

We deliberately reduce the mechanics to one dimension. The robot begins approximately facing a broad, stationary cardboard wall, moves forward only, and measures while stopped. There is no steering controller, encoder feedback, arm, object recognition or autonomous reverse recovery. Wheel mismatch, angled echoes, slipping and moving targets can invalidate this reduction.



The engineer's deliverable is a measured mapping from command to travel and a trustworthy distance observation. **The mathematician's deliverable** is an estimator, decision rule, assumptions and experiment definitions that use that mapping. The software–hardware interface is therefore explicit and testable.

Do this in the tutorial: Chapter 01, Understand the finished project; chapter 07 explains the physical interface. See the browser chapter or PDF tutorial.

2 Variables, units and the complete internal state

Use centimetres and seconds throughout this implementation. When integrating with an SI-based robotics stack, use $d_\text{m} = d_\text{cm}/100$ and $P_{\text{m}^2} = P_{\text{cm}^2}/10^4$.

Symbol	Meaning	Unit
n	Observation/decision index; increments even when stopped	dimensionless
d_n	True sensor-face-to-wall distance, hidden from controller	cm
z_n	Raw ultrasound-derived reading before bias correction	cm
$b, \hat{b}$	Actual additive sensor bias and calibrated estimate	cm
$y_n = z_n - \hat{b}$	Corrected reading	cm
μ_n, P_n	Estimated mean and variance after accepted observation	cm, cm^2
μ_n^-, P_n^-	Prior mean and variance before that observation	cm, cm^2
R, Q	Assumed measurement variance and variance added per pulse	cm^2
P_min	Numerical uncertainty floor	cm^2

Symbol	Meaning	Unit
$\hat{v}, \tau, \hat{c}$	Calibrated speed, pulse duration and coast distance	cm/s, s, cm
m	Additional heuristic movement margin	cm
κ	Context-dependent uncertainty multiplier	dimensionless
a_n	SAMPLE, MOVE, DONE or ABORT	event

The Gaussian pair is not the entire controller state. The software additionally stores observation counters, move count, verification count, a stop latch for the baselines, a progress anchor, terminal status and the first observation time. Denote this augmented state by

$$S_n = (\mu_n, P_n, N_s, N_m, N_v, N_i, N_r, L, A, T, t_0).$$

Here N_s counts observations since the last move, N_i invalid observations and N_r rejected innovations; L is the stop latch and A the progress anchor. Decisions depend on these variables as well as the belief. Thus the controller is Markovian in S_n given the next reading and time, rather than in distance belief alone.

3 From ultrasound time to a distance observation

An ultrasonic echo measures a round trip. If sound speed is c_{sound} and the measured duration is t_e, the ideal distance is

$$d = \frac{c_{\text{sound}} t_e}{2}. \tag{2}$$

The adapter uses a nominal conversion

$$z_n \text{ [cm]} = \frac{t_e \text{ [}\mu\text{s]}}{58}. \tag{3}$$

For example, 1450 μs gives 25 cm. This factor is a starting calibration convention, not a temperature compensation model. Trigger P13 is held low for 5 microseconds and high for 10 microseconds; P14 is timed with `time_pulse_us(pin14,1,12000)`. The MicroPython API reports negative values for timeout conditions [2]. The 12,000-microsecond timeout applies to the API's waiting/timing phases, not a certified upper bound on the entire sensing routine.

The adapter returns no reading for a nonpositive duration or a converted value outside 2–150 cm. The controller then applies its stricter 12–150 cm raw acceptance interval. Consequently, a physical distance below the adapter's 2 cm limit can appear as a missing observation, rather than as an immediate numeric clearance violation. This is one reason not to interpret these guards as independent obstacle protection.

The working observation model for accepted readings is

$$z_n = d_n + b + \nu_n, \qquad y_n = d_n + (b - \hat{b}) + \nu_n, \qquad \nu_n \overset{\text{model}}{\sim} \mathcal{N}(0, R). \tag{4}$$

The estimator assumes the residual bias $b - \hat{b}$ is negligible and noise is independent of prior state error. Dropouts and large outliers are handled by guards outside this Gaussian model. An angled wall or distance-dependent bias is not repaired by averaging more measurements.

Do this in the tutorial: Chapter 09, First experiment: measure without moving. See the browser chapter or PDF tutorial.

4 The movement model and its physical meaning

Let $u_n = 1$ for an executed forward pulse and $u_n = 0$ for stationary sampling. A local physical model is

$$d_{n+1} = d_n - u_n(v\tau_n + c) + w_n. \tag{5}$$

Here v is effective forward speed under the selected PWM, c is travel after motor-off, and w_n represents movement error. This simple relation is appropriate only over the calibrated operating range. It is not a full DC motor dynamics model.

For a successfully completed pulse, the implemented prediction is

$$\mu_{n+1}^- = \mu_n - \hat{v}\tau_{\text{actual}} - \hat{c}, \tag{6}$$

$$P_{n+1}^- = P_n + Q. \tag{7}$$

With defaults, $\hat{v} = 8$, $\tau = 0.12$ and $\hat{c} = 0.15$, expected travel is 1.11 cm. $Q = 0.64$ cm^2 is added once per pulse, regardless of the measured pulse duration. For SAMPLE, the state is predicted unchanged: there is no added stationary process noise. This assumes the robot and wall do not drift while waiting.

The decision rule predicts travel using configured $\tau = 0.12$ s, whereas `record_move` uses measured elapsed adapter time. That elapsed time includes some driver and stop-call overhead; it is not encoder-measured powered travel. The adapter limits requested pulses to 250 ms and PWM values to 80, checks button B during the pulse, and calls motor-off in a `finally` block. The controller accepts a reported duration up to 300 ms. These different limits accommodate overhead; they do not establish an exact bound on real travel.

After a pulse, the main program waits 200 ms before measuring. The settling time must exceed observed coast time. Failure to settle invalidates the stopped-measurement model. On cancellation or exception, the trial exits instead of resuming estimation from an unmodelled partial pulse.

Do this in the tutorial: Chapter 10, First movement: one supported-wheel pulse; then chapter 11 for floor calibration. See the browser chapter or PDF tutorial.

5 Deriving the scalar Bayesian update

Before an accepted reading, represent distance uncertainty by the working belief

$$q_n^-(d) = \mathcal{N}(d; \mu_n^-, P_n^-).$$

Under the bias-corrected likelihood $p(y_n \mid d) = \mathcal{N}(y_n; d, R)$, Bayes' rule gives

$$q_n(d) \propto \exp\left[-\frac{1}{2}\left\{\frac{(d-\mu_n^-)^2}{P_n^-} + \frac{(y_n - d)^2}{R}\right\}\right]. \tag{8}$$

Collecting the quadratic and linear terms in d yields

$$\widetilde{P}_n = \left(\frac{1}{P_n^-} + \frac{1}{R}\right)^{-1}, \qquad \mu_n = \widetilde{P}_n\left(\frac{\mu_n^-}{P_n^-} + \frac{y_n}{R}\right). \tag{9}$$

Equivalently, with Kalman gain K_n,

$$K_n = \frac{P_n^-}{P_n^- + R}, \tag{10}$$

$$\mu_n = \mu_n^- + K_n(y_n - \mu_n^-), \tag{11}$$

$$P_n = \max\{P_{\min}, (1 - K_n)P_n^-\}. \tag{12}$$

The scalar linear-Gaussian update is a special case of recursive linear filtering [1]; the variance floor and rejection rules are project-specific engineering modifications. Once those modifications or biased/correlated observations apply, the stored Gaussian is an approximate belief, not an exact posterior with guaranteed coverage.

Initialisation. There is no encoded start-position prior. The first accepted reading sets $\mu_0 = y_0$ and $P_0 = R$. It is not processed a second time as an independent measurement.

Stationary sampling. Without the floor, j further independent readings give $P_j = (1/P_0 + j/R)^{-1}$. With $P_0 = R$, this is $R/(j+1)$. The floor $P_{\min} = 0.36$ prevents a stored standard deviation below 0.6 cm. The mean continues updating after the floor is reached: the gain remains positive. This is not bias learning.

Interpretation. A large P_n^- makes a new measurement more influential; a large R makes it less influential. R and P are variances, not standard deviations. Entering a 2 cm standard deviation as $R = 2$ instead of $R = 4$ changes the controller.

Do this in the tutorial: Chapter 13, Understand the model you are running. See the browser chapter or PDF tutorial.

6 Observation guards and their exact scope

Before the update, `Controller.observe` applies these tests in order:

1. If a terminal status already exists, return it unchanged. Otherwise initialise t_0 on the first call and abort if $t - t_0 \geq 60$ s.
2. Increment N_s. A missing, nonfinite or out-of-range raw value increments N_i and clears verification. A finite raw value below 12 cm aborts immediately. Other invalid readings cause SAMPLE, or ABORT when $N_i \geq 3$ or $N_s > 16$.
3. Correct the bias. A corrected value below 12 cm aborts immediately. There is no second upper-range check after bias correction in the current implementation.
4. If a belief exists, reject the observation when
$$|y_n - \mu_n^-| > \max\left\{8,\ 4\sqrt{P_n^- + R}\right\}\ \text{cm}, \tag{13}$$
where all numeric terms are evaluated in centimetres. Increment N_r, clear verification, and SAMPLE; ABORT at $N_r \geq 3$ or $N_s > 16$.
5. On an accepted reading, reset both N_i and N_r and perform the update.

These failure counters reset on an *accepted* observation. Alternating invalid and rejected readings need not be three identical consecutive errors to trigger one of their limits. Invalid and rejected branches do not update the belief. Because they use $N_s > 16$ while the accepted branch later uses $N_s \geq 16$, a run of mixed unusable observations can reach a seventeenth observation before that particular limit aborts.

The innovation gate protects against isolated implausible returns but can also reject a real abrupt change. It is a failure detector, not a method of tracking arbitrary moving obstacles.

7 Verification: what DONE means mathematically

For Mini, define the working interval
$$I_n = [\mu_n - \kappa\sqrt{P_n},\ \mu_n + \kappa\sqrt{P_n}]. \tag{14}$$

The interval is contained in the target band exactly when

$$|\mu_n - d_\star| + \kappa\sqrt{P_n} \le \varepsilon. \tag{15}$$

To see this, require its lower endpoint to be at least $d_\star - \varepsilon$ and its upper endpoint at most $d_\star + \varepsilon$, then combine the two inequalities.

Normal context uses $\kappa = 1.64$ and careful context $\kappa = 2.33$. For a genuinely calibrated Gaussian these intervals contain approximately 89.9% and 98.0% probability respectively. The corresponding *one-sided* tail probabilities are approximately 5.05% and 0.99%. Do not confuse a central interval probability with a one-sided tail probability.

If the Gaussian assumption holds, the exact target probability is

$$p_{\mathcal{T}} = \Phi\left(\frac{29 - \mu_n}{\sqrt{P_n}}\right) - \Phi\left(\frac{21 - \mu_n}{\sqrt{P_n}}\right). \tag{16}$$

Interval containment is a sufficient condition for $p_{\mathcal{T}} \ge 2\Phi(\kappa) - 1$ under that model, not a necessary condition. The firmware evaluates the simpler inequality, not Φ.

Every successful verification test increments N_v. The first two return SAMPLE; the third returns DONE. An unsuccessful verification, rejected/invalid observation or completed movement clears N_v. Repeated posteriors share history, so three confirmations are not three independent safety certificates and their probabilities must not be multiplied.

At the variance floor, the maximum allowed mean error is $4 - 0.6\kappa$: 3.016 cm in normal mode and 2.602 cm in careful mode. Thus the target is a band, not a demand to reach exactly 25 cm.

Do this in the tutorial: Chapter 14, Understand the four events; chapter 18 explains independent measurements. See the browser chapter or PDF tutorial.

8 The implemented Mini movement decision

Verification is attempted before movement. If it has not succeeded, the algorithm applies limits and constructs

$$\Delta = \hat{v}\tau + \hat{c}, \qquad L_n = \mu_n - \kappa\sqrt{P_n} - \Delta - m. \tag{17}$$

For Mini, a move is permitted only when

$$L_n \ge 21 \text{ cm} \quad \text{and} \quad \mu_n > 25 \text{ cm}, \tag{18}$$

provided preceding guards and limits have passed. Otherwise it requests SAMPLE. Here $m = 0.5$ cm is an extra chosen margin. SAMPLE means motors off, wait 150 ms, and obtain another observation. It does not mean a motor micro-adjustment.

The distinction between prediction and uncertainty matters: equation (17) uses $\sqrt{P_n}$, *not* $\sqrt{P_n + Q}$. Q is added only after movement in equation (7). The rule also lacks an explicit bound on residual bias, motor timing error or coast variation. It is therefore an uncertainty-aware heuristic, not an implemented chance-constrained safety proof.

Possible future rule, not implemented. Under a Gaussian next-state model, a one-step lower-tail condition would instead take the form

$$\mu_n - \Delta - \kappa\sqrt{P_n + Q} - B - E \ge 21, \tag{19}$$

where B and E are validated bounds for residual bias and systematic travel error. Such a rule would require new calibration and policy tests; changing one square root alone would not certify a

robot. The 21 cm threshold is the near edge of the target band, not the physical collision boundary at $d = 0$.

Bounded waiting. On the accepted-observation path, if verification does not succeed and $N_s \geq 16$, ABORT. If a baseline stop latch is active, SAMPLE next; otherwise abort at $N_m \geq 80$ before considering movement. Verification comes first, so a final successful confirmation is allowed at a count boundary.

Progress check. An anchor is set at the first accepted mean. When N_m is a nonzero multiple of eight *and* $N_s = 1$ on an accepted observation, require anchor minus current mean to be at least 1 cm; otherwise ABORT. Then reset the anchor. This is a sparse check. If the first observation after the eighth pulse is invalid, that block's check is skipped because later observations have $N_s > 1$.

DONE and ABORT are absorbing states. The main program turns motors off and exits the active trial. There is no automatic reverse or restart; reposition and begin a new trial explicitly.

9 Three policies and the role of context

All three policies share the observation model, guards and limits; even the raw baseline maintains a belief for its innovation/progress guards. They differ in the decision rules below.

Policy	Stop/verification	Movement when not verifying
Raw	Latch stop once $y \leq 25$; then require $\lvert y - 25\rvert \leq 4$ for three accepted confirmations.	Forward pulses until latched or another guard/limit stops the trial.
Belief	Latch stop once $\mu \leq 25.5$; then require $\lvert\mu - 25\rvert + 1.64\sqrt{P} \leq 4$.	Forward pulses until latched or stopped; no Mini lower-bound test.
Mini	No stop latch; verify using equation (15).	Apply equations (17)–(18).

Context is selected before a trial; it is not estimated online. Only Mini's decision uses the selected κ. The belief baseline always uses 1.64, and raw ignores context. This explains duplicate baseline rows across contexts in the synthetic results.

In the main IPHOREOS vocabulary, the toy's *concept* is distance-to-target with uncertainty, *events* are the four actions, *context* sets κ, and *policy* maps augmented belief state to an event. It retains belief-dependent action and verification. It omits learned concepts, inferred contexts, task hierarchies, episodic memory and expected-free-energy optimisation.

One may interpret repeated sensing as information gathering: for the unmodified Gaussian update, its expected entropy reduction is $\frac{1}{2}\log(1 + P^-/R)$. The current program does not compute this quantity or optimise a weighted information/task objective. Calling it a full active-inference solver would overstate the implementation.

Do this in the tutorial: Chapter 15, Add a reduced context philosophy; chapter 17 compares the three versions. See the browser chapter or PDF tutorial.

10 Worked calculations you can reproduce

10.1 A prediction–measurement cycle

Suppose an accepted reading initialises $(\mu, P) = (50, 4)$. Mini requests MOVE, since $50 - 1.64(2) - 1.11 - 0.5 = 45.11 \geq 21$. After an actual 0.12 s pulse,

$$\mu^- = 48.89, \qquad P^- = 4.64.$$

For a new corrected reading of 49 cm, $K = 4.64/8.64 = 0.537037$, hence

$$\mu = 48.949074 \text{ cm}, \qquad P = 2.148148 \text{ cm}^2.$$

This is a constructed arithmetic example, not a real robot measurement.

10.2 Same measurement, different context

The first reading is 26 cm, so $\mu = 26$, $\sqrt{P} = 2$. Normal verification fails because $1 + 1.64(2) = 4.28 > 4$. But the predicted lower value is $26 - 3.28 - 1.11 - 0.5 = 21.11$, so normal requests MOVE. Careful verification also fails and its lower value is $26 - 4.66 - 1.11 - 0.5 = 19.73$, so careful requests SAMPLE. The context changes an action through a specific inequality, not an unexplained label.

10.3 Three confirmations

With three consecutive 25 cm readings in normal mode, the posterior pairs are $(25, 4)$, $(25, 2)$ and $(25, 4/3)$. Each interval fits the band. The actions are SAMPLE, SAMPLE, DONE. With no action between them, the default model adds no process noise. Careful context fails the first test $(2.33 \cdot 2 > 4)$, then requires three successful confirmations starting at the second observation.

10.4 Confidence can be wrong

If true distance is 20 cm and residual bias is +5 cm, repeated readings near 25 cm can lead to a narrow belief and DONE outside the true band. No variance floor or repetition count identifies an unmodelled constant bias by itself. The next section explains why an independent ruler reference is essential.

The supplied `tools/check_math_model.py` evaluates these examples against the actual shared controller and records numeric results in `results/math_model_checks.json`.

11 Calibration as parameter estimation

11.1 Sensor bias and measurement variance

At measured ruler distances d_i^{ref}, collect raw observations z_i and form residuals $e_i = z_i - d_i^{\text{ref}}$. The current calibration tool computes

$$\hat{b} = \bar{e} = \frac{1}{N}\sum_i e_i, \qquad \hat{R} = \max\left\{0.1, \frac{1}{N-1}\sum_i (e_i - \bar{e})^2\right\}. \tag{20}$$

The tool requires at least ten rows; the tutorial recommends 20 readings at each of 20, 30, 40 and 60 cm, with a held-out check at 45 cm. Its CSV unit is cm. Pooling is justified only if residual

bias/variance are reasonably stable over distance. Plot residuals by distance and order; a trend calls for a better calibration model, and serial correlation calls for more conservative uncertainty treatment.

Ruler alignment and reference uncertainty contribute to these residuals. If independent reference error has known variance R_{ref}, the residual variance contains both sensor and reference contributions; the provided tool does not separate them. The standard error $\sqrt{\hat{R}/N}$ describes bias estimation only under independent, representative samples, and the firmware does not automatically add that bias uncertainty to P.

11.2 Travel per pulse

For repeated trials under fixed PWM/floor/battery conditions, record powered time τ_i, powered travel ℓ_i, and coast c_i. The tool reports

$$\hat{v} = \frac{1}{J}\sum_i \frac{\ell_i}{\tau_i}, \qquad \hat{c} = \frac{1}{J}\sum_i c_i, \qquad \hat{Q} = \text{sampleVar}(\ell_i + c_i). \tag{21}$$

It also reports maximum observed coast. At least five trials are required; more repetitions across expected battery conditions give a more useful estimate. The pooled travel variance is meaningful for a fixed pulse duration. If duration varies, a preferable future estimate is residual variance about a fitted travel model, rather than variance of mixed-duration distances. That regression is not implemented by this tool.

If powered travel and coast cannot be separated reliably, use total travel divided by the fixed pulse duration as effective speed and set coast to zero, as described in the tutorial. Do not also add coast, which would double-count it. This effective model is local to the calibrated pulse duration.

11.3 Identifiability and model checking

From stationary data $z = d + b + \nu$ alone, the pair (d, b) is unidentifiable: replacing it by $(d+\delta, b-\delta)$ leaves the likelihood unchanged. Repeated samples improve precision about $d + b$, not identification of d. Likewise, one fixed-duration travel experiment identifies $v\tau + c$, not v and c separately. Independent distance references and either coast observations or varied-duration experiments resolve those ambiguities.

Choose $P_{\min}$, m and operating limits conservatively from held-out behaviour; the tool does not infer them. Its JSON includes metadata as well as parameters. Transfer only relevant parameter fields to `CALIBRATION`, rather than passing the entire JSON to `Controller`, which rejects unknown keys. Neither the tool nor this document enables motors automatically.

Do this in the tutorial: Chapter 11, Calibrate the physical model; chapter 12 enters and checks the estimates. See the browser chapter or PDF tutorial.

12 From the mathematical action to motor power

For a differential-drive robot with wheel radius r, wheel separation B_w and angular speeds ω_L, ω_R, ideal no-slip kinematics give

$$v = \frac{r}{2}(\omega_L + \omega_R), \qquad \dot{\theta} = \frac{r}{B_w}(\omega_R - \omega_L). \tag{22}$$

The one-dimensional approximation assumes heading error remains small and $\dot{d} \approx -v$. Matching numeric PWM values does not guarantee equal wheel speeds. Use supported-wheel tests and

measured straight travel to assess the assumption; the current code has no feedback correction for heading.

Command chain. The event MOVE reaches `Robot.pulse(left,right,milliseconds)`. It calls the vendor driver for left and right forward motor operation, waits briefly while checking B, and switches both channels off. The on-board motor electronics supply energy from the batteries. The mathematical variable $\hat{v}$ is a calibrated consequence of that chain; sending the number 8 to a motor API would not mean 8 cm/s.

PWM 45 is a provisional vendor-scale command (0–255), while this project's training cap is 80. It is not directly a voltage measurement or a guaranteed duty-cycle/speed relation. Load, supply voltage, friction and controller electronics intervene. Use the kit's integrated driver rather than supplying motors from micro:bit signal pins.

Mathematical operation	Software location	Engineering evidence
$z = t_e/58$	`firmware/hardware.py`, distance	Known-distance readings
μ, P update and guards	`belief_controller.py` in `firmware/`, observe	Desktop examples and ruler errors
Event to pulse	`firmware/main.py`, MOVE branch	Supported wheels, measured duration
μ^-, P^- prediction	Controller, record_move	Measured travel/coast
Calibration values	`firmware/config.py`	Recorded CSVs and held-out checks
Independent truth in experiments	`simulate.py`, or physical ruler	Never reuse estimate as ground truth

Logs are emitted before the chosen action is executed. A MOVE row therefore records the pre-movement posterior, not the new predicted position. Hardware time is emitted in milliseconds and simulation time in seconds; convert before comparing. Software printing does not by itself create a stored onboard experiment file.

PicoGK is a geometry kernel for computational engineering [4]. A later design could translate bracket dimensions, sensor clearances and enclosure constraints into geometry. Our proposed use is an optional mechanical design extension; it is not part of this kit's runtime control path. Changing geometry would require revisiting sensor reference position and motion calibration.

Do this in the tutorial: Chapter 07, See exactly how software moves hardware; chapter 08 covers programming. See the browser chapter or PDF tutorial.

13 Default parameters: a complete reproducible specification

Configuration / quantity	Default	Interpretation
`target_cm`, `tolerance_cm`	25, 4 cm	Nominal goal and half-band
`minimum_cm`, `maximum_cm`	12, 150 cm	Controller raw limits
`sensor_bias_cm`	0 cm	Placeholder until calibrated
`measurement_var`	4 cm^2	Assumed observation variance
`variance_floor`	0.36 cm^2	Minimum stored variance
`motion_var`	0.64 cm^2	Added variance per pulse
`speed_cm_s`, `coast_cm`	8 cm/s, 0.15 cm	Provisional motion model
`extra_margin_cm`	0.5 cm	Heuristic next-step margin
`pulse_s` / `PULSE_MS`	0.12 s / 120 ms	Decision/requested pulse
`SETTLE_MS`	200 ms	Post-pulse waiting

Configuration / quantity	Default	Interpretation
`SAMPLE_INTERVAL_MS`	150 ms	Stationary observation delay
`max_samples`	16	Accepted-path stationary limit
`max_invalid`	3	Invalid counter threshold
Innovation rejection threshold	3	Hard-coded rejected count limit
`max_moves`	80	Pulse-count limit
`timeout_s`	60 s	From first observe call
`verify_count`	3	Target confirmations
Normal / careful κ	1.64 / 2.33	Mini context multiplier
`PWM_LEFT`, `PWM_RIGHT`	45, 45	Provisional vendor commands
`MAX_PWM`	80	Project cap; adapter also caps 80
`MOTORS_ENABLED`	False	Explicit preparation gate
`CALIBRATION_CONFIRMED`	False	Explicit run-mode gate

The controller validates distance ordering and several positive parameters, but not every possible malformed configuration. Keep counters positive integers and all calibration numbers finite; calibration output is not a substitute for inspecting input data. `SENSOR_MAX_CM` in the configuration is descriptive in this version: the hardware adapter's 150 cm cap is literal. Changing that setting alone does not change the adapter.

14 Simulation: what the reported results actually measure

The supplied experiment runs 300 seeds for each of three policies, two contexts and two scenarios: $300 \cdot 3 \cdot 2 \cdot 2 = 3600$ trials. Initial distance is uniform on 45–85 cm, true speed uniform on 6.8–9.2 cm/s. Nominal bias is uniform on $[-0.6, 0.6]$ cm; stress bias is fixed at +5 cm. The controller retains its uncalibrated default bias zero in both scenarios.

At each observation, independent Gaussian noise has standard deviation 1.6 cm. A uniform draw determines a 2% dropout, a disjoint 1% +12 cm outlier, or an ordinary reading. True movement is

$$d_{n+1} = d_n - \max\{0, 0.12v + 0.15 + \eta_n\}, \qquad \eta_n \sim \mathcal{N}(0, 0.20^2). \tag{23}$$

The controller still uses $R = 4$ and $Q = 0.64$, so its uncertainty assumptions are intentionally different from the nominal generator. Separate random streams govern sensing and motion. Same-seed trials share initial conditions, but different action sequences produce different physical observation paths.

Simulation time increases by 0.32 s for MOVE and 0.15 s for SAMPLE. Sensor/API/printing overhead and actual measured pulse timing are omitted. These elapsed times cannot be treated as hardware benchmarks. A simulation-only 600-iteration cap complements the controller limits.

For trial i, define

$$S_i = \mathbf{1}\{T_i = \text{DONE},\ |d_i^{\text{final}} - 25| \le 4\}, \tag{24}$$

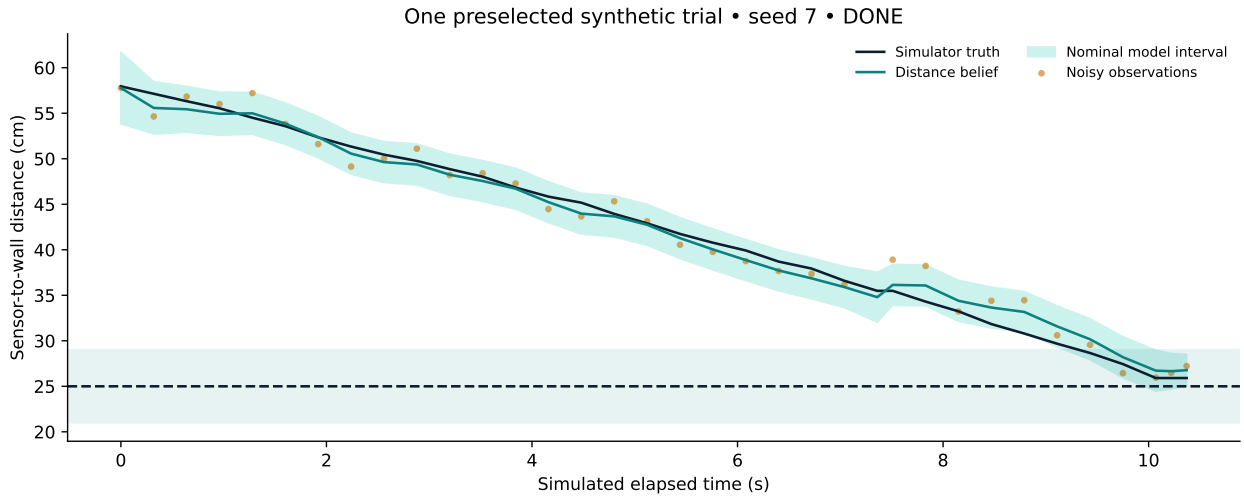
$$F_i = \mathbf{1}\{T_i = \text{DONE},\ |d_i^{\text{final}} - 25| > 4\}, \tag{25}$$

$$A_i = \mathbf{1}\{T_i = \text{ABORT}\}, \qquad E_i = |d_i^{\text{final}} - 25|. \tag{26}$$

Report rates as averages across *all* trials. False verification rate $\bar{F}$ is not the conditional fraction $\sum F_i / \sum \mathbf{1}\{T_i = \text{DONE}\}$; label either denominator explicitly. Mean absolute error includes aborted trials in the supplied summary. Wall contacts count simulated move steps for which $d \le 0$, rather than distinct physical collision episodes.

Scenario	Policy/context	Success	False verified	Aborted
Nominal	Raw/normal	98.67%	0.33%	1.00%
Nominal	Belief/normal	99.00%	0.00%	1.00%
Nominal	Mini/normal	99.33%	0.00%	0.67%
Nominal	Mini/careful	99.33%	0.00%	0.67%
+5 cm bias	Raw/normal	18.00%	81.00%	1.00%
+5 cm bias	Belief/normal	9.00%	90.33%	0.67%
+5 cm bias	Mini/normal	72.00%	27.33%	0.67%
+5 cm bias	Mini/careful	34.67%	64.67%	0.67%

These are descriptive synthetic results from the supplied CSV, not evidence that a purchased robot achieves these rates. Increasing κ need not improve performance under misspecification: verification and action timing both change, while bias remains. There is no claimed statistical superiority between policies.



One preselected synthetic trace, seed 7. The shaded plot uses $1.96\sqrt{P}$ for display; the normal controller uses $1.64\sqrt{P}$ in its decisions. The plotted interval is not empirically calibrated coverage.

Do this in the tutorial: Chapter 19, Run the complete project on your laptop; chapter 20 interprets results. See the browser chapter or PDF tutorial.

15 Physical experiment design and acceptance evidence

Run stationary checks first, then supported wheels, then short floor pulses, then autonomous trials. Keep the broad cardboard wall, start range, floor and sensor reference consistent. Rotate policy order to avoid confounding policy with declining battery voltage. Record every attempt, including upload/runtime failures, aborts and user stops; do not silently remove inconvenient trials.

Measure final sensor-face distance independently with a ruler. Record policy, context, start/final distance, terminal event, time and any wall contact in `data/physical_trials_template.csv`. An operator stop is not a verified success; record it explicitly in notes and define its treatment before comparing results.

At least a small repeated pilot is needed to find mechanical problems; it does not establish high reliability. For an empirical success proportion $\hat{p} = s/N$, an approximate sampling standard error is $\sqrt{\hat{p}(1-\hat{p})/N}$ under independent trials, but this estimate is misleading at zero/all successes. For example, with zero failures in N independent representative trials, the exact one-sided 95% upper failure probability solves $(1-p)^N = 0.05$, giving $p = 1 - 0.05^{1/N}$. This limits what a perfect small pilot can establish. Correlated trials and untested floors weaken even that interpretation.

Evaluate distance errors and failure types, not only completion rate. A model can be computationally correct but physically wrong because of biased sensing, wheel slip, timing or unsuitable geometry. Recalibrate after changing PWM, pulse duration, floor, batteries or sensor mounting.

No physical validation has been performed for this package. The 18 existing software tests and added mathematical checks verify stated computations and selected behaviours, not real-world safety or micro:bit resource compatibility.

Do this in the tutorial: Chapter 16, Run your first floor trial; chapters 17–18 compare and record experiments. See the browser chapter or PDF tutorial.

16 Reading route, source files and tutorial crosswalk

Mathematical question	This companion	Tutorial chapter (PDF page)
What is success?	Sections 1–2	01 (page 2)
What does the sensor measure?	Section 3	09 (page 10)
How does a pulse move the robot?	Sections 4, 12	07, 10 (pages 8, 11)
How is uncertainty updated?	Section 5	13 (page 14)
When should it stop or sample?	Sections 6–8	14 (page 15)
How does context matter?	Section 9	15 (page 16)
How do I check the arithmetic?	Section 10	13, 19 (pages 14, 20)
Where do the numbers come from?	Sections 11, 13	11–12 (pages 12–13)
How do I evaluate the result?	Sections 14–15	17–20 (pages 18–21)

Practical links: PDF tutorial; browser tutorial; offline replay; project README.

Implementation authority: the files in `firmware/`: `belief_controller.py`, `hardware.py`, `main.py` and `config.py`. The seven generated `flash/` files embed these sources. Synthetic results come from `simulate.py`; calibration summaries from `tools/calibrate.py`. This document describes the current implementation rather than adding a different controller.

Rebuild/check: from the project directory, run the following commands. Run the second command twice to resolve the PDF cross-references.

```
python tools/check_math_model.py
pdflatex -interaction=nonstopmode -halt-on-error Mathematical_Model.tex
```

The project archive includes editable LaTeX, numeric check output, the tutorial and its assets.

References

[1] R. E. Kalman (1960). A New Approach to Linear Filtering and Prediction Problems. *Journal of Basic Engineering* 82(1), 35–45. doi:10.1115/1.3662552. Background for the scalar Gaussian recursion; project-specific guards and policy are derived above.

[2] BBC micro:bit MicroPython V2 documentation. *Machine: time_pulse_us*. https://microbit-micropython.readthedocs.io/en/v2-docs/machine.html. Accessed 10 September 2026.

[3] Kitronik. *:MOVE Motor 5683*. https://kitronik.co.uk/products/5683-kitronik-move-motor-for-the-bbc-micro-bit. Driver source and retained licence are included in the project; see `docs/vendor_manifest.json` for pinned hashes. Physical calibration remains required.

[4] LEAP 71. *PicoGK: geometry kernel for computational engineering*. https://picogk.org/. Accessed 10 September 2026. The suggested bracket/enclosure use is our proposed extension.